PES UNIVERSITY

Department of Computer Science and Engineering
PES University, Bangalore, India

UE23CS341B: Compiler Design

Prakash C O, Associate Professor, Department of CSE

Compiling a C Program: Language Processing System

GNU Compiler Collection (GCC):

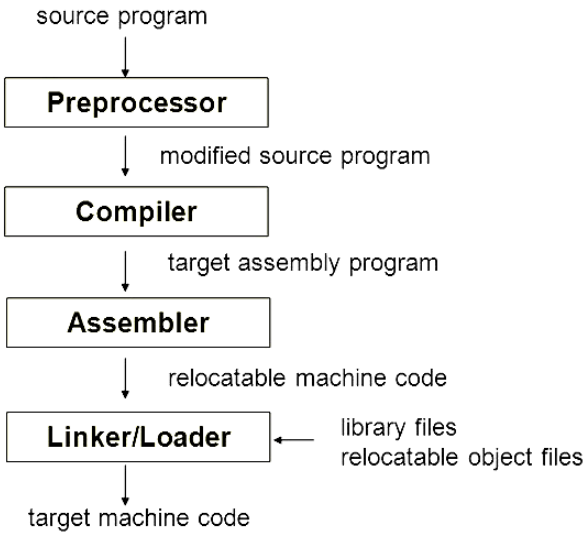
- **GCC stands for "GNU Compiler Collection". GCC is an integrated distribution of compilers for several major programming languages**. These languages currently include C, C++, Objective-C, Objective-C++, Fortran, Java, Ada, D, Go, and BRIG (HSAIL).
- GCC is a free software project that includes compilers for Ada, C, C++, Fortran, Java, and Objective-C, as well as libraries for these languages.
- **GCC is capable of generating executables for a variety of platforms including x86, ARM, MIPS, PowerPC, etc.**

Note:

- GNU stands for Gnu's Not Unix. It is a recursive acronym, and it stands for "Gnu's Not Unix". GNU is a free and open-source operating system that was started in 1984 by Richard Stallman.
  - **Linux is the kernel:** the program in the system that allocates the machine's resources to the other programs that you run. The kernel is an essential part of an operating system, but useless by itself; it can only function in the context of a complete operating system. Linux is normally used in combination with the GNU operating system: the whole system is basically GNU with Linux added, or GNU/Linux. All the so-called "Linux" distributions are really distributions of GNU/Linux.]

- GCC is written in C with a strong focus on portability, and can compile itself, so it can be adapted to new systems easily.
- **GCC is a portable compiler it runs on most platforms available today**, and can produce output for many types of processors.
- **GCC is not only a native compiler it can also *crosscompile* any program**, producing executable files for a different system from the one used by GCC itself.
- **GCC has multiple language *frontends*, for parsing different languages**.
  The GCC includes front ends for C, C++, Objective-C, Fortran, Ada, Go, and D, as well as libraries for these languages (libstdc++,...).
  Programs in each language can be compiled, or crosscompiled, for any architecture.
  For example, an ADA program can be compiled for a microcontroller, or a C program for a supercomputer.

Compiling a C Program: Language Processing System



MinGW - Minimalist GNU for Windows

- **A native Windows port of the GNU Compiler Collection (GCC)**
- **https://sourceforge.net/projects/mingw/**

```
C:\MinGW\bin\gcc - demo>gcc -E hello.c > hello.i

C:\MinGW\bin\gcc - demo>gcc -S hello.i

C:\MinGW\bin\gcc - demo>as hello.s -o hello.o

C:\MinGW\bin\gcc - demo>gcc hello.o

C:\MinGW\bin\gcc - demo>.\a
Hello, world!

C:\MinGW\bin\gcc - demo>
```

Input Program: <hello.c>

```
#include <stdio.h>
int main (void)
{
      printf ("Hello, world!\n");
      return 0;
}
```

The below section describes in detail how GCC transforms source files to an executable file.

Compilation is a multistage process involving several tools, including

- the GNU Compiler itself (through the ***gcc*** or g++ frontends),
- the GNU Assembler ***as***, and
- the GNU Linker ***ld***.

The complete set of tools used in the compilation process is referred to as a *toolchain*.

Stages of Compilation

The sequence of commands executed by a single invocation of GCC consists of the following stages:

1) **Preprocessing (cpp)**
   - Handles directives like #include, #define, and conditional compilation (#ifdef).
   - Expands macros and includes header files.
   - Output: a "pure" source file with all macros expanded (still in C/C++).
2) **Compilation**
   - Translates preprocessed source into **assembly language** for the target architecture.
   - Performs syntax analysis, optimization, and code generation.
   - Output: assembly file (.s)
3) **assembly (as)**
   - Converts assembly instructions into **machine code** (binary object file).
   - Output: object file (.o)
4) **linking (ld)**
   - Combines object files and static/dynamic libraries.
   - Resolves symbol references (functions, variables).
   - Produces the final executable (a.out by default)

Step-I: The preprocessor

The first stage of the compilation process is the **use of the preprocessor to expand macros** and **included header files.**

```
$ cpp hello.c > hello.i
  or
$ gcc -E hello.c > hello.i
```

The result is a file 'hello.i' which contains the source code with all macros expanded.

By convention, preprocessed files are given the file extension '.i' for C programs and '.ii' for C++ programs.
In practice, the preprocessed file is not saved to disk unless the savetemps option is used or you redirect the output to an output file.

Step-II: The compiler

The next stage of the process is the actual compilation of preprocessed source code to assembly language, for a specific processor.

```
$ gcc -S hello.i
```

The resulting assembly language code is stored in the file 'hello.s'.

```
csestaff@pesu-Inspiron-3576:~$ cat hello.s
        .file   "hello.c"
        .section        .rodata
.LC0:
        .string "Hello, world!"
        .text
        .globl  main
        .type   main, @function
main:
.LFB0:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    $.LC0, %edi
        call    puts
        movl    $0, %eax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size   main, .-main
        .ident  "GCC: (Ubuntu 5.4.0-6ubuntu1~16.04.11) 5.4.0 20160609"
        .section        .note.GNU-stack,"",@progbits
```

Explanation:

1) ***.rodata*** indicates read only data section.
2) ***LC0*** is a label that contains the read only data of type string which is "Hello, world!\n".
3) ***.text*** section contains the executable code.
4) ***.globl*** main indicates main is visible globally
5) ***.type main, @function*** indicates main is a global function.
6) The registers are:
    a. ***rbp*** (stack base pointer)
    b. ***rsp*** (stack pointer)
7) base pointer is pushed to the stack and the value is moved to rsp register.
8) ***call puts*** indicates a call to the external function printf/puts

Step III: The assembler

- The purpose of the assembler is to convert assembly language into machine code and generate an object file. When there are calls to external functions in the assembly source file, the assembler leaves the addresses of the external functions undefined, to be filled in later by the linker.
- Developers and compiler writers can inspect the generated assembly to verify correctness, analyze performance, or debug issues.

The assembler can be invoked with the following command line:

```
$ as hello.s -o hello.o
```

As with GCC, the output file is specified with the o option. The resulting file 'hello.o' contains the machine instructions for the *Hello World* program, with an undefined reference to printf.

Step IV: The linker

- The final stage of compilation is the linking of object files to create an executable.
- In practice, an executable requires many external functions from system and C runtime (crt) libraries. Consequently, the actual link commands used internally by GCC are complicated.
  For example, the full command for linking the *Hello World* program is:

  $ **ld** -dynamic-linker /lib/ld-linux.so.2 /usr/lib/crt1.o/usr/lib/crti.o /usr/lib/gcc-lib/i686/3.3.1/crtbegin.o -L/usr/lib/gcc-lib/i686/3.3.1 hello.o -lgcc -lgcc_eh -lc -lgcc -lgcc_eh /usr/lib/gcc-lib/i686/3.3.1/crtend.o/usr/lib/crtn.o

  Fortunately, there is never any need to type the command above directly the entire linking process is handled transparently by gcc when invoked as follows:

```
$ gcc hello.o
```

This links the object file 'hello.o' to the C standard library, and produces an executable file 'a.out'

Step IV: The loading

```
$ ./a.out
```

Hello, world!

This loads the executable file into memory and causes the CPU to begin executing the instructions contained within it.

Symbol table

A symbol table is metadata inside object files (.o) and executables (a.out, ELF binaries, etc.).

- **It records:**
  - Function names and their addresses.
  - Global/static variables and their addresses.
  - Section references (like .text, .data, .bss).
- **Used by:**
  - The linker to resolve references between object files.
  - The debugger to map machine addresses back to human-readable names.
  - Tools like nm or objdump to inspect compiled binaries.

Executables and object files can contain a symbol table. This table stores the location of functions and variables by name, and can be displayed with the nm command as follows:

$ nm a.out

The nm command lists symbols from object files and executables. For example:

```
csestaff@pesu-Inspiron-3576:~$ nm a.out
0000000000601038 B __bss_start
0000000000601038 b completed.7594
0000000000601028 D __data_start
0000000000601028 W data_start
0000000000400460 t deregister_tm_clones
00000000004004e0 t __do_global_dtors_aux
0000000000600e18 t __do_global_dtors_aux_fini_array_entry
0000000000601030 D __dso_handle
0000000000600e28 d _DYNAMIC
0000000000601038 D _edata
0000000000601040 B _end
00000000004005b4 T _fini
0000000000400500 t frame_dummy
0000000000600e10 t __frame_dummy_init_array_entry
00000000004006f8 r __FRAME_END__
0000000000601000 d _GLOBAL_OFFSET_TABLE_
                 w __gmon_start__
00000000004005d4 r __GNU_EH_FRAME_HDR
00000000004003c8 T _init
0000000000600e18 t __init_array_end
0000000000600e10 t __init_array_start
00000000004005c0 R _IO_stdin_used
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
0000000000600e20 d __JCR_END__
0000000000600e20 d __JCR_LIST__
                 w _Jv_RegisterClasses
00000000004005b0 T __libc_csu_fini
0000000000400540 T __libc_csu_init
                 U __libc_start_main@@GLIBC_2.2.5
0000000000400526 T main
                 U puts@@GLIBC_2.2.5
00000000004004a0 t register_tm_clones
0000000000400430 T _start
0000000000601038 D __TMC_END__
```

The output shows that the start of the main function has the hexadecimal offset 0000000000400526. Most of the symbols are for internal use by the compiler and operating system.

The most common use of the ***nm*** command is to check whether a library contains the definition of a specific function, by looking for a 'T' entry in the second column against the function name.

Symbol Types in nm Output

Each line shows:

- **Address:** Where the symbol is located.
- **Type code:** What kind of symbol it is.
- **Name:** The symbol's identifier.

Common type codes:

- T → Symbol in the text (code) section (global function).
- t → Local (static) function in text section.
- B → Uninitialized global variable (in .bss).

- D → Initialized global variable (in .data).
- U → Undefined symbol (needs to be resolved by the linker).
- R → Read-only data.

Finding dynamically linked libraries

The command ***ldd*** examines an executable and displays a list of the shared libraries that it needs. These libraries are referred to as the shared library *dependencies* of the executable.

$ldd a.out

```
csestaff@pesu-Inspiron-3576:~$ ldd a.out
        linux-vdso.so.1 =>  (0x00007ffc4ff6d000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f8b5f3c7000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f8b5f791000)
```

The output shows that the program depends on the C library **libc** (shared library version 6) and the dynamic loader library **ld-linux** (shared library version 2).

Static and Dynamic Libraries

- When a C program is compiled, the compiler generates object code. After generating the object code, the compiler also invokes linker.
- **One of the main tasks for linker is to make code of library functions (eg printf(), scanf(), sqrt(), ..etc) available to your program**.
- A linker can accomplish this task in two ways, by copying the code of library function to your object code, or by making some arrangements so that the complete code of library functions is not copied, but made available at run-time.

A static library is a collection of compiled object files bundled together. When linked statically, the code from the library is copied into the final executable.

Static Linking and Static Libraries is the result of the **linker making copy of all used library functions to the executable file.**
Static Linking creates larger binary files, and need more space on disk and main memory.
Examples of static libraries (libraries which are statically linked) are .a files on Linux/Unix systems and .lib files on Windows. These files contain compiled code that is linked directly into an executable at build time, making the program self-contained without needing external library files at runtime.

Following are some important points about static libraries.

1) For a static library, the actual code is extracted from the library by the linker and used to build the final executable at the point you compile/build your application.
2) Each process gets its own copy of the code and data. Where as in case of dynamic libraries it is only code shared, data is specific to each process. For static libraries memory footprints are larger. For example, if all the window system tools were statically linked, several tens of megabytes of RAM would be wasted for a typical user, and the user would be slowed down by a lot of paging.
3) Since library code is connected at compile time, the final executable has no dependencies on the library at run time i.e. no additional run-time loading costs, it means that you don't need to

carry along a copy of the library that is being used and you have everything under your control and there is no dependency.
4) In static libraries, once everything is bundled into your application, you don't have to worry that the client will have the right library (and version) available on their system.
5) One drawback of static libraries is, for any change(up-gradation) in the static libraries, you have to recompile the main program every time.
6) One major advantage of static libraries being preferred even now "is speed". There will be no dynamic querying of symbols in static libraries. Many production line software use static libraries even today.

When to Use Static Libraries

- Embedded development: Where runtime dynamic linking is not feasible.
- Portable software distribution: Ensures the program runs without missing dependencies.
- Security-sensitive applications: Reduces risk of dependency hijacking by bundling all code.

Dynamic linking and Dynamic Libraries

Dynamic Linking doesn't require the code to be copied, it is done by just placing name of the library in the binary file. The actual linking happens when the program is run, when both the binary file and the library are in memory.

Examples of Dynamic libraries (libraries which are linked at run-time) are, ***.so*** in Linux and ***.dll*** in Windows and **.dylib** in **macOS.**

Advantages of Dynamic Linking

- **Reduced executable size**: Code is not duplicated in every program.
- **Memory efficiency**: Multiple programs can share the same library loaded in memory.
- **Easy updates**: Updating the library automatically benefits all programs using it.
- **Flexibility**: Libraries can be swapped or upgraded without recompiling the program.

Role of the Linker in Dynamic Linking

In dynamic linking, the linker plays a crucial role in preparing a program to use shared libraries at runtime rather than embedding all code into the executable itself.

1. **Symbol resolution (at compile/link time)**
    - The linker does not copy the actual library code into the executable.
    - Instead, it records references to external functions or variables (symbols) that reside in shared libraries (like .dll in Windows or .so in Linux).
2. **Stub creation**
    - **The linker inserts *stubs* or *placeholders* in the executable**.
    - These stubs tell the operating system loader which dynamic libraries are needed and where to find the required functions.
3. **Relocation information**
    - The linker generates relocation tables so that, when the program is loaded, the operating system can adjust memory addresses to point to the actual library functions.
4. **Library dependency management**

- The linker embeds metadata in the executable that specifies which dynamic libraries must be loaded at runtime.

- For example, a program may depend on `libc.so` (the standard C library) — the linker ensures this dependency is recorded.

What Happens at Runtime

- When the program starts, the loader (part of the OS) reads the linker's metadata.
- The loader finds and loads the required shared libraries into memory.
- It resolves the stubs created by the linker, binding them to the actual addresses of the library functions.
- The program can then call those functions as if they were part of its own code.

Benefits of Dynamic Linking

- Reduced executable size: Code is not duplicated in every program; shared libraries are reused.
- Easy updates: Updating a shared library automatically benefits all programs that use it.

Memory efficiency: Multiple programs can share the same library code in memory.

- Flexibility: Programs can load libraries conditionally or at runtime (e.g., plugins).

References:

- https://www.geeksforgeeks.org/compiling-a-c-program-behind-the-scenes/
- https://gcc.gnu.org/onlinedocs/gcc-9.2.0/gcc.pdf
- 02. Lang Processing System(Hands-on), Prof. Preet Kanwal, PESU.

What is relocatable machine code?

Relocatable Machine Code refers to machine code that can be loaded into **any memory location** and executed correctly without modification. Unlike absolute machine code, which is tied to a fixed memory address, relocatable code uses **relative addressing** or symbolic references that can be adjusted during the linking or loading phase.

Key Features

- **Not fixed to a specific address** in memory.
- Uses **relative addresses** or symbolic names for variables and functions.
- The **linker/loader** adjusts addresses when the program is loaded into memory.

Why Important?

- Allows **modular programming** (multiple object files combined).
- Supports **dynamic loading** and **shared libraries**.
- Enables **efficient memory management** in operating systems.

Example

- Object files generated by compilers are usually **relocatable**.
- When linking, the linker resolves addresses and produces an **absolute executable**.

Object Files and Relocatable Code

- When a compiler translates source code, it usually produces **object files** (e.g., .o or .obj files).

- These object files contain:
  - **Machine instructions** for the program.
  - **Symbol table** (names of functions, variables).
  - **Relocation information** (places where addresses need adjustment).
- The machine instructions in object files are **relocatable**, meaning they do not assume a fixed memory address. Instead, they use **relative addresses** or symbolic references.

Why Relocatable?

- Programs are often split into multiple modules (files).
- Each module is compiled separately into an object file.
- At this stage, the compiler does not know the final memory layout of the entire program.
- So, addresses for functions and variables are left **symbolic** or **relative**.

Role of the Linker

- The **linker** combines multiple object files into a single **executable file**.
- It:
  - Resolves **symbolic references** (e.g., function calls across files).
  - Adjusts addresses based on the final memory layout.
  - Produces **absolute machine code** (fixed addresses) for execution.

Example

Suppose you have:

- main.o calls func() defined in utils.o.
- In main.o, the call to func() is stored as a **symbolic reference**.
- The linker finds func() in utils.o, assigns it an address (say 0x400500), and updates the call instruction in main.o to point to that address.

Summary

- **Object files** = relocatable machine code (addresses not fixed yet).
- **Linker** = resolves addresses and produces **absolute executable code** ready for loading and running.

Symbolic Reference

- A **symbolic reference** is a placeholder or name used in the object file to represent an address or location that is **not yet known** at compile time.
- Instead of storing the actual memory address of func(), the compiler stores its **symbol name** (e.g., "func") in the object file.
- This is necessary because:
  - At compile time, the compiler does not know where func() will be located in memory.
  - The final memory layout is determined later by the **linker** when combining multiple object files.

How It Works

- In main.o, the call instruction might look like:

```
CALL func
```

Here, func is a **symbolic reference**, not an actual address.

- The **linker** resolves this symbolic reference by:
  - Finding the definition of func() in another object file (e.g., utils.o).
  - Assigning a real memory address (e.g., 0x400500).
  - Updating the call instruction to:

CALL 0x400500

Why Important?

- Enables **modular compilation** (compile files separately).
- Supports **relocatable code**, so programs can be loaded at different memory locations.
- Makes linking and dynamic loading possible.